7. Lectura i escriptura de fitxers

2025-2026

Continguts

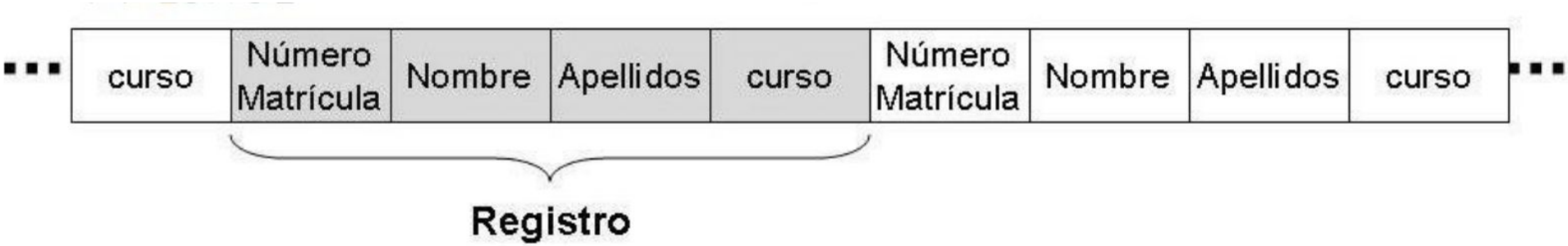
- Concepte Registre.
- Fluxos de dades.
- Classes relatives a fluxes.
- Utilització de fluxos.
- Fitxers de dades.
- Persistència i Serialització.

1. Concepte Registre

"Un registre és una **agrupació de dades**."

Cadascuna d'aquestes dades o elements es denomina **camp**.

Aquests camps poden ser tipus primitius o altres estructures de dades.



2. Fluxos de dades

Els fluxos de dades permeten enviar o rebre informació entre el programa i l'origen o destí de la informació, independentment del orígen.

Els programes solen llegir i escriure emprant fluxos de dades sense importar a on es llegeixen o s'escriuen les dades. Això els permet simplificar la programació d'aquesta transferència de dades.

En Java s'implementen els fluxos de dades mitjançant objectes stream.
(Exemple: **InputStream** o **OutputStream**)

2. Classes relatives a fluxos

El paquet **java.io** proporciona classes per l'entrada i sortida de dades mitjançant diferents tipus de variables. Es una part fundamental del API de Java per aquestes operacions de I/O.

Tipus de dades que permet emprar (cada un amb una classe diferent):

- byte[]
- char[]
- File (Arxius i directoris)
- Fluxos de dades

3. Fitxers de dades

Formes d'accés a fitxers de dades:

- **Accés seqüencial**. L'accés al registre *n* implica la lectura prèvia dels registres del 1 al *n*-1
- **Accés directe**. Els registres s'accedeixen expressant la seva direcció en el fitxer
- **Accés per índex**. L'accés a les dades es fa mitjançant una clau. La clau es cerca a una taula, la qual té associats clau i direcció relativa als registres. Una vegada que es coneixen la direcció relativa s'accedeix a les dades.
- **Accés dinàmic**. Es pot accedir a les dades mitjançant qualsevol de les formes anteriorment esmentades.

Escriptura seqüencial a un Fitxer

```java
public static void escribirArxiu() throws IOException{
    FileOutputStream file = null;
    char caracter;
    String text = "En una galaxia muy muy lejana...";
    try{
        file = new FileOutputStream("datos.txt");
        for(int i=0; i<text.length(); i++){
            caracter = text.charAt(i);
            file.write((byte) caracter);
        }
    }catch(IOException e){
        e.printStackTrace();
    } finally {
        file.close();
    }
}
```

Lectura seqüencial a un Fitxer

```java
public static void llegirArxiu() throws IOException{
    FileInputStream file = null;
    String text = "";
    char caracter;
    try{
        file = new FileInputStream("datos.txt");
        int size = file.available();
        for(int i=0; i<size; i++){
            caracter = (char) file.read();
            text = text+caracter;
        }
    }catch(IOException e){
        e.printStackTrace();
    } finally {
        file.close();
    }
}
```

La classe File

La classe File pertany a la classe pare Object.

Recorda

L’objecte *File* pot treballar tant a fitxers com a directoris

Constructor de File

Constructors	Descripció
public File (String ruta_absoluta)	Aquest constructor crea un objecte File al qual l'hem de passar tota la ruta inclòs el nombre d'arxiu
public File (String ruta, String nom)	Aquest constructor l'hem de passar la ruta relativa i el nom del fitxer
public File (File ruta, String nom)	Com l'objecte File pot representar un directori o un fitxer, el primer paràmetre funcionarà com directori (ruta relativa) i el segon paràmetre serà el nom del fitxer a crear.

Exemple de creació de File

```
File f1 = new File("/home/datos.txt");

File f2 = new File("/home", "datos.txt");

File dir = new File("/home");
File f3 = new File(dir, "datos.txt");
```

Mètodes útils de File

Mètodes	Descripció
public boolean isDirectory() public boolean isFile() public boolean exists()	Per comprovar si el objecte File existeix o per saber si és un Directori o un Fitxer.
public boolean delete()	Permet esborrar el fitxer o directori.
public boolean canRead() public boolean canWrite()	Indiquen si tenim o no permisos de lectura o d'escriptura damunt el Fitxer.
public String getPath() public String getAbsolutePath()	Retorna la ruta relativa o absoluta de l'arxiu (El absolut inclou el nom del fitxer)

Mètodes útils de File

Mètodes	Descripció
public String getName()	Retorna el nom de l'arxiu.
public long length()	Retorna el tamany de l’arxiu en bytes.
public boolean mkdir()	Crea un nou directori.
public String[] list() public String[] list(filtre)	Retorna un array de Strings amb els noms dels fitxers/directoris del directori on apunta el objecte File.
public File[] listFiles()	Retorna un array d’objectes File amb els fitxers/directoris del directori on apunta.

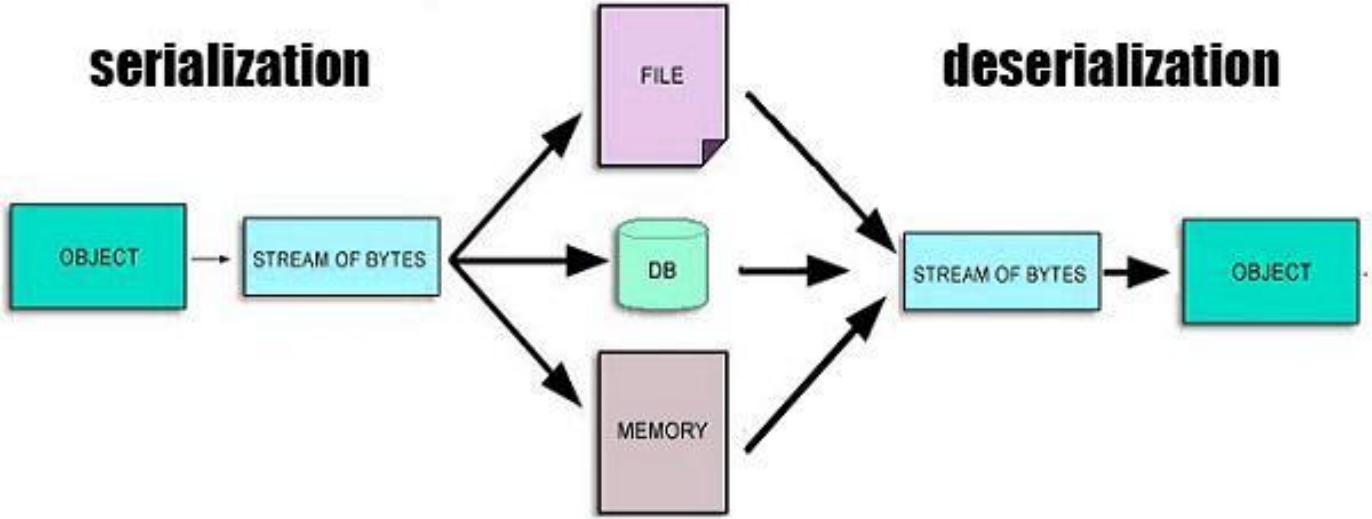
5. Persistència i Serialització

Aquests termes ens permeten emmagatzemar objectes dins fitxers.

- **Persistir** es un terme que empram per **indicar que s'han desat els canvis**, ja sigui dins un fitxer o dins la base de dades.

- **Serialitzar** consisteix en **codificar els objectes** com una secuencia de bytes per poder persistirlos.

Serialitzar i deserialitzar objectes

La passa contraria a la serialització es denomina deserialització.

Si per exemple, tenim l'objecte serialitzat i emmagatzemat dins un fitxer, seria posible recompondre l'objecte Java mitjançant la deserialització.